ECE C147/C247, Winter 2024 **Midterm Solution**
Neural Networks & Deep Learning Prof. J.C. Kao
UCLA ECE TAs: T. Monsoor, Y. Liu, S. Rajesh, L. Julakanti, K. Pang

UCLA True Bruin academic integrity principles apply.
Open: Four pages of cheat sheet allowed, Calculator.
Closed: Book, computer, Internet.
2:00pm-3:50pm PST.
Wednesday, 21 Feb 2024.

State your assumptions and reasoning.
No credit without reasoning.
Show all work on these pages.

Name: _____

Signature: _____

ID#: _____

Problem 1	_____ / 20
Problem 2	_____ / 35
Problem 3	_____ / 20
Problem 4	_____ / 25
BONUS	_____ / 7 bonus points
Total	_____ / 107 points

1. **Multiple choice and short answer.** (20 points)

(a) (3 points) You are tasked with training a 2 layer Fully Connected Neural Network (FCNN) with batch normalization and dropout. Which of the following are hyperparameters?

i. Loss function
ii. Dropout probability p
iii. Batchnorm rescaling factors γ and β
iv. Batch size

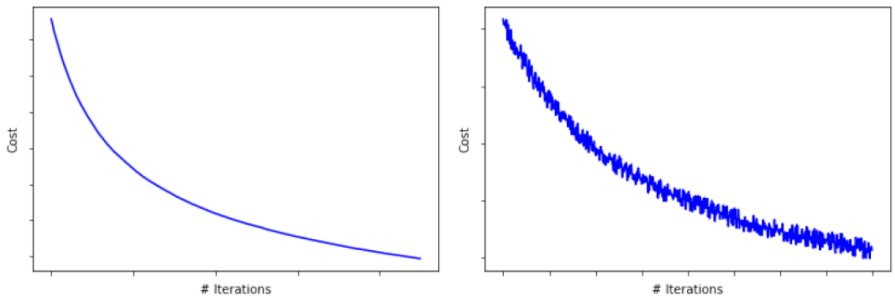
Solution: i, ii and iv since Batchnorm Parameters are learnt using gradient descent.

(b) (3 points) Which of the following are valid techniques to perform regularization?

i. Early Stopping.
ii. Norm Penalties.
iii. Model Ensembling.
iv. Dropout.

Solution: i, ii, iii and iv. All of the above are regularization techniques.

(c) (3 points) Consider the training loss curves, shown in Fig. 1, for two different optimization routines. In one routine we estimate the gradient using a Batch and in the other we estimate the gradient using a Minibatch . Based on these graphs, select which of the following are true.



(a) Graph A (b) Graph B

Figure 1: Loss trajectory of two different optimization routines

i. Graph A represents Minibatch Gradient Descent since the gradient estimates using batch and mini-batch are equivalent.
ii. Graph A represents Batch Gradient Descent because it estimates the gradient accurately, resulting in a decrease in loss at every iteration.
iii. Graph B represents Minibatch Gradient Descent since it is a noisy estimate of the gradient, resulting in oscillations in the loss trajectory.

Solution: ii and iii since minibatch gradient descent is a noisy estimate of the gradient each update may not necessarily results in the loss moving down.

(d) (3 points) Select the true statements below regarding k-fold cross validation.

i. We evaluate the cross validation performance as the mean of the performance on the validation set of the k different folds.

ii. We evaluate the cross-validation performance as the best one over k different folds.

iii. In a k-fold cross validation approach, k different model instances are fitted.

iv. In a k-fold cross validation approach, the same model instance is applied to evaluate the validation performance of all folds.

Solution: i and iii. K-fold cross validation averages out performance on each fold and fits K different models and not one model one after the other.

(e) (5 points) Your friend encounters problems when using a Fully Connected Neural Network (FCNN) for a deep learning task and asks for your help.

i. (2 points) Your friend is confused about whether to choose ReLU or PReLU as the activation function. Below are the equations for these two activation functions, where α is a learnable parameter.

$$\text{ReLU}(x) = \max(0, x), \qquad \text{PReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

Please explain one advantage of PReLU over ReLU in no more than three sentences.
Solution: PReLU introduces a learnable parameter α for negative inputs. This avoids the stopping of learning when the input $x < 0$ where ReLU may suffer from zero-gradient issue. PReLU can also mitigate the zig-zag issue. Moreover, as α is optimized during training, it allows the model to adaptively select the most effective activation function.

ii. (3 points) In the experiment, the FCNN model achieves a low training error and a high test error. Your friend thinks this is due to underfitting and plans to add more layers to increase the model capacity. Do you agree with his evaluation? Justify your answer and, if you disagree with your friend, provide two suggestions to improve model performance.
Solution: Disagree. The low training error and high test error of the model indicates overfitting, and further adding more layers can make overfitting more serious.
Suggestions: reducing model complexity/increasing dataset size/adding L-1 or L-2 regularization terms/batchnorm/adding dropout layers/data augmentation/soft label/early stopping...

(f) (3 points) Suppose you are deploying Convolutional Neural Networks (CNNs) for image classification. Describe two advantages of a CNN over an FCNN in no more than 3 sentences.

Solution: CNN outperforms FCNN in image classification due to the use of convolutional layers, which are able to capture effective spatial features in images, like edges and shapes. Convolutional layers share parameters and hence CNNs require fewer parameters than FCNNs, making them more efficient in terms of memory and time, and less likely to suffer from overfitting, particularly within large-scale images.

2. **Hybrid norm constraints in Similarity Neural Network (SNN)** (35 points)

A similarity neural network (SNN) consists of twin networks which accept distinct inputs but share the same weights. The outputs of the twin networks are later processed by more hidden layers. Let's assume we have a SNN with the following architecture:

$$
\begin{aligned}
h_p &= W_1 x_p^{(i)} + b_1 \\
z_1 &= \text{softplus}(h_p) \\
h_q &= W_1 x_q^{(i)} + b_1 \\
z_2 &= \text{softplus}(h_q) \\
z &= z_1 - z_2 \\
z_3 &= W_2 z + b_2 \\
\hat{y}^{(i)} &= \sigma(z_3) \\
L^{(i)} &= L_{CE}(y^{(i)}, \hat{y}^{(i)}) \\
L &= -\frac{1}{m} \sum_{i=1}^{m} L^{(i)}
\end{aligned}
$$

In the above architecture, $(x_p^{(i)}, x_q^{(i)})$ represent the pair of ith input example and are each of shape $\mathbb{R}^{D_x \times 1}$. $y^{(i)}$ represent the label of the ith input example and is a scalar, $y^{(i)} \in \{0, 1\}$. We also assume z_1 and z_2 have shape of $\mathbb{R}^{D_z \times 1}$. $\sigma(\cdot)$ denotes the sigmoid function. L_{CE} denotes the binary cross entropy loss. Softplus is a smooth approximation to the ReLU function and can be used to constrain the output to always be positive. It is an element-wise function defined as follows:

$$\text{softplus}(x) = \frac{1}{\beta} \log(1 + e^{\beta x})$$

For your convenience, we have drawn out the computational graph for you in Figure 2

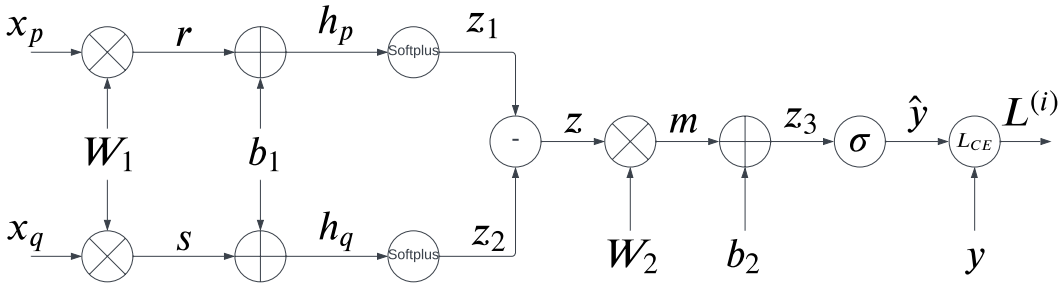


Figure 2: Computational graph for SNN

(a) (3 points) Compute $\nabla_{\hat{y}^{(i)}} L^{(i)}$ and denote it as $\delta_{\hat{y}^{(i)}}$. For all the following parts, you can refer to this computed gradient as $\delta_{\hat{y}^{(i)}}$.

Hint: Recall the binary cross entropy loss is:

$$L^{(i)} = y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Solution: From the computational graph,

$$L^{(i)} = y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Since all quantities are scalar, so by product rule of derivatives:

$$\delta_{\hat{y}^{(i)}} = \frac{y^{(i)}}{\hat{y}^{(i)}} - \left[\frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}}\right]$$

(b) (3 points) Compute $\nabla_{z_3} L^{(i)}$ and denote it as δ_{z_3}. For all the following parts, you can refer to this computed gradient as δ_{z_3}.

Solution: From the computational graph,

$$\hat{y}^{(i)} = \sigma(z_3).$$

Hence, by scalar chain rule

$$\delta_{z_3} = \sigma(z_3)[1 - \sigma(z_3)]\delta_{\hat{y}^{(i)}}$$

(c) (3 points) Compute $\nabla_{W_2} L^{(i)}$ and denote it as δ_{W_2}. For all the following parts, you can refer to this computed gradient as δ_{W_2}.

Solution: From the computational graph,

$$m = W_2 z.$$

Hence, by chain rule

$$\delta_{W_2} = \delta_{z_3} z^T.$$

(d) (3 points) Compute $\nabla_z L^{(i)}$ and denote it as δ_z. For all the following parts, you can refer to this computed gradient as δ_z.

Solution: From the computational graph,

$$m = W_2 z.$$

Hence, by chain rule

$$\delta_z = \delta_{z_3} W_2^T.$$

(e) (4 points) Compute $\nabla_{h_q} L^{(i)}$ and denote it as δ_{h_q}. For all the following parts, you can refer to this computed gradient as δ_{h_q}.

Solution: From the computational graph,

$$z_2 = \text{softplus}(h_q)$$

Since softplus like *ReLU* is an elementwise operation so backpropagating through it leads to a hadamard product

$$\delta_{h_q} = \left(\frac{e^{\beta h_q}}{1 + e^{\beta h_q}} \right) \odot \left(- \delta_z \right)$$

(f) (4 points) Compute $\nabla_{h_p} L^{(i)}$ and denote it as δ_{h_p}. For all the following parts, you can refer to this computed gradient as δ_{h_p}.

Solution: From the computational graph,

$$z_1 = \text{softplus}(h_p)$$

Since $Softplus$ like *ReLU* is an elementwise operation so backpropagating through it leads to a hadamard product

$$\delta_{h_p} = \left(\frac{e^{\beta h_p}}{1 + e^{\beta h_p}} \right) \odot \left(\delta_z \right)$$

(g) (2 points) Compute $\nabla_{b_1} L^{(i)}$.

Solution: By law of total derivatives,

$$\nabla_{b_1} L^{(i)} = \delta_{h_p} + \delta_{h_q}$$

(h) (5 points) Compute $\nabla_{W_1} L^{(i)}$.

Solution: By law of total derivatives and using the 3-D tensor derivative trick,

$$\nabla_{W_1} L^{(i)} = \delta_{h_p} {x_p^{(i)}}^T + \delta_{h_q} {x_q^{(i)}}^T$$

(i) (8 points) Based on some prior knowledge, you want to impose some norm constraints on W_1 and W_2. After exploring the various norm constraints, you have decided to modify the loss function in the following way:

$$\tilde{L} = \left(- \frac{1}{m} \sum_{i=1}^{m} L^{(i)} \right) + LSE(W_2) + \|W_1\|_\infty,$$

where Log-Sum-Exponent (LSE) of a vector $x \in \mathbb{R}^n$ is defined as follows:

$$LSE(x) = \ln \left(\sum_{i=1}^{n} e^{x_i} \right)$$

Compute $\nabla_{W_1}\tilde{L}$ and $\nabla_{W_2}\tilde{L}$.

Solution: Since gradient is a linear operator, so

$$\nabla_{W_1}\tilde{L} = -\frac{1}{m}\sum_{i=1}^{m}\nabla_{W_1}L^{(i)} + \nabla_{W_1}\|W_1\|_\infty$$
$$\nabla_{W_2}\tilde{L} = -\frac{1}{m}\sum_{i=1}^{m}\nabla_{W_2}L^{(i)} + \nabla_{W_2}LSE(W_2)$$

Now, let j denote the index of the row of W_1 with maximum row sum, then

$$\nabla_{W_1}\|W_1\|_\infty = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \text{sign}(w_j) \\ \mathbf{0} \end{bmatrix}$$

Similarly,

$$\begin{aligned}\nabla_{W_2}LSE(W_2) &= \begin{bmatrix} \frac{e^{x_1}}{\sum_{i=1}^{D_z} e^{x_i}} \\ \vdots \\ \frac{e^{x_{D_z}}}{\sum_{i=1}^{D_z} e^{x_i}} \end{bmatrix}^T \\ &= \text{softmax}(W_2)\end{aligned}$$

Hence,

$$\nabla_{W_1}\tilde{L} = -\frac{1}{m}\sum_{i=1}^{m}\nabla_{W_1}L^{(i)} + \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \text{sign}(w_j) \\ \mathbf{0} \end{bmatrix}$$
$$\nabla_{W_2}\tilde{L} = -\frac{1}{m}\sum_{i=1}^{m}\nabla_{W_2}L^{(i)} + \text{softmax}(W_2)$$

3. **Regularization techniques** (20 points)

(a) (7 points) Derive the gradient descent step for linear regression where $\mathcal{L}(\theta) = \frac{1}{2}(\mathbf{y} - \mathbf{X}\theta)^T(\mathbf{y} - \mathbf{X}\theta)$ where $\mathbf{y} \in \mathbb{R}^N$, $\mathbf{X} \in \mathbb{R}^{N\times 2}$, and $\theta \in \mathbb{R}^2$.
Hint: $\frac{\partial \mathbf{z}^T\theta}{\partial\theta} = \mathbf{z}$ and $\frac{\partial\theta^T\mathbf{A}\theta}{\partial\theta} = (\mathbf{A} + \mathbf{A^T})\theta$.
Solution:

$$\begin{aligned}
\mathcal{L}(\theta) &= \frac{1}{2}(\mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\theta + \theta^T\mathbf{X}^T\mathbf{X}\theta) \text{ (3 points)} \\
\nabla_\theta\mathcal{L}(\theta) &= \frac{\partial}{\partial\theta}\mathcal{L}(\theta) \\
&= \frac{1}{2}(-2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\theta) \text{ (2 points)} \\
&= -\mathbf{X}^T\mathbf{y} + \mathbf{X}^T\mathbf{X}\theta \text{ (1 point)} \\
\theta &\leftarrow \theta - \epsilon\nabla_\theta\mathcal{L}(\theta) \\
&\leftarrow \theta - \epsilon(-\mathbf{X}^T\mathbf{y} + \mathbf{X}^T\mathbf{X}\theta) \text{ (1 point)}
\end{aligned}$$

(b) (5 points) Derive the gradient descent step for ridge regression where $\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\alpha}{2}||\theta||_2^2$.
Hint: You can leave your final answer in terms of $\nabla_\theta\mathcal{L}(\theta)$.
Solution:

$$\begin{aligned}
\tilde{\mathcal{L}}(\theta) &= \mathcal{L}(\theta) + \frac{\alpha}{2}\theta^T\theta \text{ (2 points)} \\
\nabla_\theta\tilde{\mathcal{L}}(\theta) &= \frac{\partial}{\partial\theta}\tilde{\mathcal{L}}(\theta) \\
&= \nabla_\theta\mathcal{L}(\theta) + \alpha\theta \text{ (2 points)} \\
\theta &\leftarrow \theta - \epsilon\nabla_\theta\tilde{\mathcal{L}}(\theta) \\
&\leftarrow \theta - \epsilon[\nabla_\theta\mathcal{L}(\theta) + \alpha\theta] \\
&\leftarrow (1 - \epsilon\alpha)\theta - \epsilon\nabla_\theta\mathcal{L}(\theta) \text{ (1 point)}
\end{aligned}$$

(c) (4 points) Explain one regularizing effect of ridge regression on a model. Your answer should not exceed three sentences.
Solution:
As shown in the previous part, ridge regression shrinks the weight. (2 points)
The weight decay leads to smaller weights in the final model, which tends to prevent overfitting. (2 points)

(d) (4 points) You implemented inverted dropout in Homework 4. What is the reasoning behind dividing the activations by p, where p is the probability of keeping a neuron, in the forward pass of training? Your answer should not exceed three sentences.
Solution:
At test time, we do not perform dropout since we want to take advantage of the entire network. (1 point)
However, we still want to keep the activations on roughly the same scale between training and testing. (1 point)

Therefore, we need to upscale, or equivalently divide by p, the activations in the forward pass of training since we are only using a portion of the neurons in the network. (2 points)

4. **Optimization techniques** (25 points)

(a) (7 points) Consider the loss function with a local minima as shown in Figure 3. We assume we started at the initial position labeled 'Start' and arrived at a local minima after 5 iterations of vanilla gradient descent as shown in Figure 3. We have accumulated some momentum along the way.

After arriving at the local minima, we are considering the next gradient descent step, which we will take with the accumulated momentum. We have two options: Option 1: GD + momentum, and Option 2: GD + Nesterov momentum. Between Option 1 and 2, which optimizer will result in a weight update with larger magnitude? Explain your answer in no more than 4 sentences. If your answer depends on the accumulated momentum, state why that is in your answer.

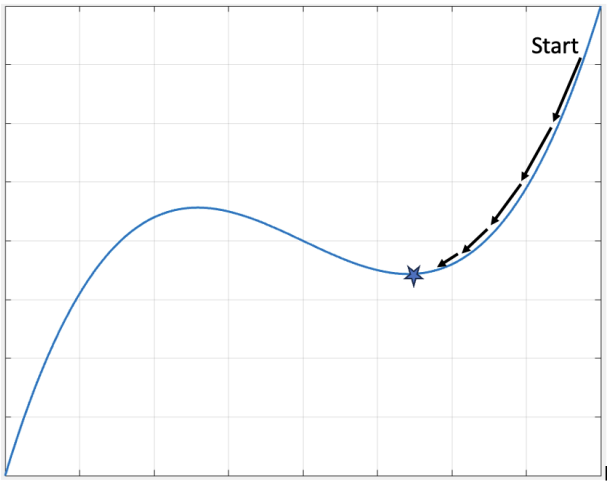


Figure 3: Loss function $L(x)$

Solution: The weight updates in the next iteration will be according to the following equations for GD+momentum and GD+Nesterov momentum:

$$\text{GD+momentum update: } v_{t+1} = \alpha v_t - \epsilon * \nabla_\theta L(\theta_t)$$
$$\text{GD+Nesterov momentum update: } v_{t+1} = \alpha v_t - \epsilon * \nabla_\theta L(\theta_t + \alpha v_t)$$
$$\text{Weight updates: } \theta_{t+1} = \theta_t + v_{t+1}$$

The GD+momentum will have a gradient of zero at the local minima and the weight update will only depend on the momentum. So, the gradient of Nesterov momentum will determine which one has the greater magintude weight update.

If the momentum accumulated over the downwards slope is enough to surpass the local maxima in the plot, the nesterov momentum will have a greater magnitude as the gradient will add up to momentum and will further push the optimizer out of the minima.

If the momentum accumulated is not enough to surpass the local maxima in the plot, the the nesterov momentum will have a lesser magnitude as the gradient acts opposite to the momentum and decreases the magnitude of weight update.

(b) (10 points) Consider the contour plot shown in Figure 4 , where the level sets of the loss surface is plotted with respect to the 2 weights X and Y, where $X, Y \in R$ (scalars). From the contour plot, we can tell that the shown region contains a "valley". Assume we are given a hypothetical scenario: we start from point 1 and use vanilla gradient descent for many iterations to get to point 2. During this process, we accumulated momentum (first moment) and second moments. Now carefully observe the plot, and answer the following questions for the next few iterations of weight updates.

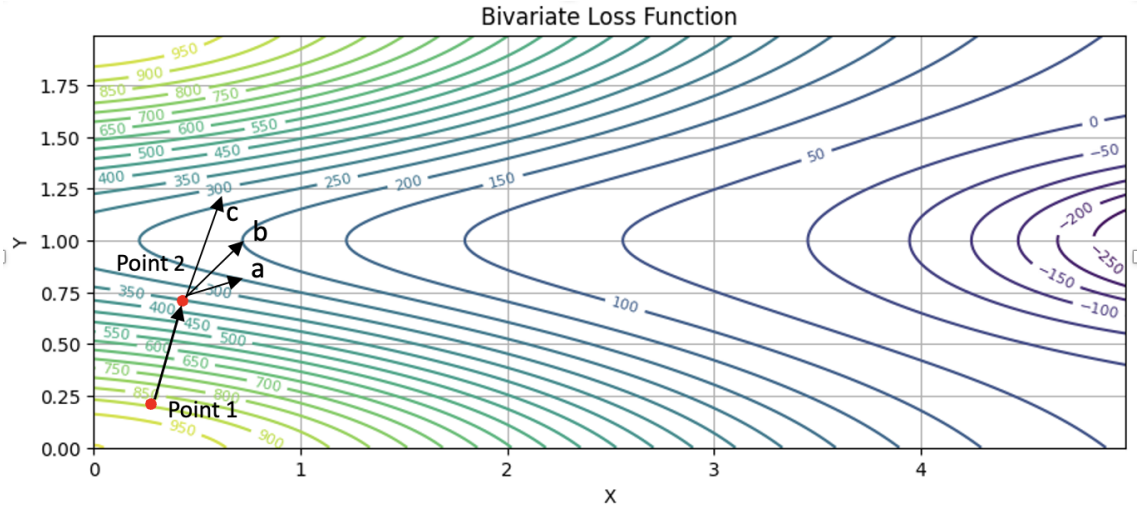


Figure 4: Contour plot of a Loss function $L(X, Y)$

i. (7 points) The update steps a, b and c in the contour plot, correspond to three different algorithms based on the same history trajectory, are plotted. Please match the steps with the following three optimization algorithms: GD + Momentum, Adagrad and Adam. Justify your reason in no more than 6 sentences.

Solution: We can see from contour plot that at Point 2, we have larger gradient in the y direction than in the x direction. Since we know that weight updates in "GD with momentum" is governed by the direction with larger gradient so step c corresponds to update of GD+Momentum.
In Adagrad, the learning rate along the direction of the larger gradient is smaller and the weight update in the direction of smaller gradient is larger. In our case, the step a corresponds to the update of Adagrad.
Adam optimizer combines the Adagrad with momentum and hence, its update would not be dominantly x or y. So, the step b corresponds to update of Adam.

ii. (3 points) Which of the above three algorithms will face issue in escaping the valley in the next few iterations ? Explain the reason for this issue.

Solution: The GD+Momentum will face issue in escaping the valley because of the zigzagging nature over the next few iterations as the momentum and the gradient will be in opposing directions.

(c) (8 points) In the Gradient descent + Nesterov momentum scheme, find a general expression for θ_t (weight parameter) in terms of θ_0 (initial weights), the gradients $g_1, g_2, ..., g_t$, α (momentum hyperparameter) and ϵ(learning rate), considering an initial value of momentum $v_0 = 0$. Consider the following equations for the weight update in Nesterov momentum:

$$v_t = \alpha v_{t-1} - \epsilon g_t$$
$$\theta_t = \theta_{t-1} + v_t + \alpha(v_t - v_{t-1})$$

Solution: For GD+ Nesterov momentum we have the following update rules,

$$v_t = \alpha v_{t-1} - \epsilon g_t$$
$$\theta_t = \theta_{t-1} + v_t + \alpha(v_t - v_{t-1})$$

From the above equations, we can obtain the following equations after recursions and substituting $v_0 = 0$

$$v_t = -\epsilon \sum_{i=1}^{t} \alpha^{t-i} g_i \text{ (3 points)}$$

$$\theta_t = \theta_0 + \alpha(v_t - v_0) + \sum_{j=1}^{t} v_j \text{ (4 points)}$$

$$\theta_t = \theta_0 - \epsilon\alpha \sum_{i=1}^{t} \alpha^{t-i} g_i - \epsilon \sum_{j=1}^{t} \left(\sum_{i=1}^{j} \alpha^{j-i} g_i \right) \text{ (1 point)}$$

5. **Bonus: Sparsity using ℓ_1 regularization** (7 points)
Please note that no partial credit is awarded for the bonus problem

In lectures and discussion, we have remarked that adding an ℓ_1 regularized term promotes sparsity in the weight vectors. In this problem, we will show the mathematical justification for the remark. Let's consider the following regualrized cost function:

$$L_{reg}(w) = L(w) + \alpha\|w\|_1.$$

Applying Taylor series expansion to $L(w)$ near the optimizer w^* and discarding higher-order terms we get

$$L(w) \approx L(w^*) + \frac{1}{2}(w - w^*)^T H(w - w^*),$$

where

$$w^* = \arg\min_w L(w)$$

and $H = diag(h_1, h_2, \cdots, h_n)$ is a diagonal matrix. You can assume $h_i > 0, \forall i$.

(a) (2 points) Use the above approximation to compute $\nabla_{w_i} L_{reg}(w)$ for $i = 1, 2, \cdots, n$.

Solution: Using the above approximation,

$$\nabla_w L_{reg}(w) = \nabla_w L(w^*) + \nabla_w[\frac{1}{2}(w - w^*)^T H(w - w^*)] + \alpha\nabla_w\|w\|_1$$

Since,

$$\nabla_w L(w^*) = 0$$

and H is a diagonal matrix so we can write the gradients in terms of w_i,

$$\nabla_{w_i} L_{reg}(w) = h_i(w_i - w_i^*) + \alpha sign(w_i), \text{for } \ i = 1, 2, \cdots, n$$

(b) (5 points) Find a piecewise solution to the following equation

$$\nabla_{w_i} L_{reg}(w) = 0,$$

for $i = 1, 2, \cdots, n$. Your solution should only be in terms of w_i^*, h_i, α.

Solution: Let

$$g(w_i) = h_i w_i + \alpha sign(w_i) - h_i w_i^*.$$

Hence, we want to find the solution to

$$g(w_i) = 0.$$

We will find the solution for different cases: If $w_i^* > 0$ and $\alpha - h_i w_i^* \leq 0$, then the solution to $g(w_i) = 0$ is given by

$$w_i = w_i^* - \frac{\alpha}{h_i}.$$

If $w_i^* > 0$ and $\alpha - h_i w_i^* > 0$, then the solution to $g(w_i) = 0$ is given by

$$w_i = 0.$$

If $w_i^* < 0$ and $-\alpha - h_i w_i^* > 0$, then the solution to $g(w_i) = 0$ is given by

$$w_i = w_i^* + \frac{\alpha}{h_i}.$$

If $w_i^* < 0$ and $-\alpha - h_i w_i^* < 0$, then the solution to $g(w_i) = 0$ is given by

$$w_i = 0.$$

Putting all the cases together, we have

$$w_i = \begin{cases} w_i^* - \frac{\alpha}{h_i}, & w_i^* \geq \frac{\alpha}{h_i} \\ 0, & \frac{-\alpha}{h_i} < w_i^* < \frac{\alpha}{h_i} \\ w_i^* + \frac{\alpha}{h_i}, & w_i^* \leq \frac{-\alpha}{h_i} \end{cases}$$

Clearly, from the above solution we can see that there is a region where w_i's are explicitly set to zero thus leading to a sparse solution.